

2025 机器学习期末复习（软件）

2311451 李宗杭

目录

一、绪论.....	2
二、线性模型.....	7
三、神经网络.....	10
四、深度学习.....	15
五、支持向量机（SVM）	20
六、机器学习模型评估	23
七、贝叶斯分类器（无大题）	25
八、网络机器学习	27
九、集成学习.....	28
十、聚类（无大题）	29
十一、机器学习理论.....	33
十二、Python 基本用法	36

一、绪论

1. 机器学习定义

定义 1: 若一段计算机程序针对任务类 T ，通过性能度量 P 评估，其性能随经验 E 的积累而提升，则称该程序从经验中学习。

例如，在垃圾邮件检测任务中，任务 T 是判断邮件是否为垃圾邮件，经验 E 是通过大量已标注的邮件数据进行训练，性能度量 P 可以是准确率（正确判断为垃圾邮件或非垃圾邮件的比例）

关键要素：任务 T （如分类、回归）、经验 E （数据）、性能度量 P （准确率、误差等）。

定义 2: 机器学习是赋予计算机无需显式编程即可学习能力的研究领域。

核心本质: 从数据中挖掘潜在规律，构建模型（如函数 $f(x)$ ）以预测未知样本。

2. 典型应用与非机器学习场景

(1) 属于机器学习的应用：

图像识别: 如 LeNet-5 网络用于手写数字识别，通过学习大量的手写数字图像及其对应的标签，能够对手写数字图像进行准确分类。CNN 处理图像分类。

自然语言处理: 如 Transformer 架构在机器翻译、文本生成等任务中的应用，通过学习大量的文本数据，能够生成自然流畅的翻译结果或文本内容。BERT 做文本理解。

自动驾驶: 通过学习车辆行驶过程中的各种传感器数据（如摄像头图像、激光雷达数据等），实现对道路环境的感知和决策，从而实现自动驾驶功能。特斯拉 FSD、华为智驾通过传感器数据学习控制策略。

AI 下棋: AlphaGo 通过强化学习掌握围棋策略。

(2) 不属于机器学习的场景：

异或逻辑: 虽需模型，但可通过显式编程实现（如多层感知机）。传统的单层感知器无法解决异或问题，因为它是一个非线性问题，而单层感知器只能解决线性可分问题。机器学习中的多层神经网络可以通过引入隐藏层来解决异或问题。

太阳位置预测: 这是一个基于物理规律和天文学计算的问题，可以通过精确的数学模型和公式来计算，而不需要机器学习。机器学习适用于那些有潜在规律但难以用精确公式表达的问题。

3. 中国新一代人工智能与传统人工智能的区别

传统人工智能：依赖规则枚举（如下棋时穷举多步策略），适用于逻辑清晰的场景，这种方法在面对复杂的棋局时可能会遇到计算量过大的问题。

新一代人工智能：以机器学习为核心，通过学习大量的数据来发现数据中的潜在规律，并利用这些规律进行预测和决策。例如，在自动驾驶中，通过学习大量的驾驶场景数据，模型可以自动学习到如何在不同的路况下做出合理的驾驶决策。适用于复杂、规律难以显式描述的场景（如图像、语音）。

4. 代价函数

代价函数（也称为损失函数）是衡量模型预测值与真实值之间差异的函数。常见的代价函数包括：

均方误差（MSE）：用于回归任务，计算预测值与真实值之间差的平方的平均值。

交叉熵损失（Cross-Entropy Loss）：用于分类任务，衡量模型输出的概率分布与真实标签的概率分布之间的差异。

代价函数的作用：通过最小化代价函数，可以优化模型的参数，使模型在训练数据上的预测结果更接近真实结果。

5. 分类

（一）监督学习（Supervised Learning）

监督学习是指从标记的训练数据中学习模型，然后将模型应用于未标记的数据以进行预测。监督学习可以进一步分为以下两类：

（1）分类（Classification）

分类任务的目标是将输入数据分配到预定义的类别中。输出是离散的类别标签。常见的分类算法包括：

逻辑回归（Logistic Regression）：通过 Sigmoid 函数将线性回归的输出映射到(0, 1)区间，表示样本属于正类的概率。

医学领域中，根据患者的年龄、性别、症状等特征，预测患者是否患有某种疾病（如糖尿病、心脏病等）。例如，给定患者的血糖水平、血压、家族病史等特征，逻辑回归模型可以输出患者患病的概率，从而帮助医生进行诊断。

支持向量机（SVM）：通过寻找最大间隔超平面来划分不同类别的数据。

在图像识别领域，用于区分不同类别的图像，如识别手写数字（MNIST 数据集），将图像分为 0 到 9 的数字类别。SVM 通过寻找最优分割超平面，能够有效地将不同类别的图像分隔开来。

决策树 (Decision Trees)：通过一系列规则将数据划分为不同的类别。

在金融领域，根据客户的收入、信用评分、贷款历史等特征，决策树可以用来预测客户是否会违约。通过一系列的规则（如“如果信用评分大于 600 且收入大于 50000，则预测为不会违约”），决策树能够直观地展示决策过程。

随机森林 (Random Forest)：基于 Bagging 的集成学习方法，通过构建多个决策树并进行投票来提高分类性能。

在生态学中，用于预测植物物种的分布。给定地理位置、土壤类型、气候条件等特征，随机森林可以综合多个决策树的预测结果，提高植物物种分布的预测准确性。

朴素贝叶斯 (Naive Bayes)：基于贝叶斯定理，假设特征之间相互独立，通过计算条件概率来进行分类。

在文本分类任务中，如垃圾邮件检测。通过计算邮件内容中单词的条件概率，朴素贝叶斯模型可以判断一封邮件是否为垃圾邮件。例如，如果邮件中包含“中奖”、“免费”等词汇的概率较高，则模型可能将其分类为垃圾邮件。

K 最近邻 (K-Nearest Neighbors, KNN)：根据最近的 K 个邻居的类别进行投票，确定新样本的类别。

在推荐系统中，根据用户的历史行为（如购买的商品、浏览的网页等），KNN 可以找到与目标用户最相似的 K 个用户，然后根据这些邻居用户的偏好推荐商品或内容。

神经网络 (Neural Networks)：通过多层神经元结构学习数据中的复杂模式，适用于复杂的分类任务。

在语音识别系统中，如智能语音助手（Siri、Alexa 等）。神经网络能够学习语音信号的复杂模式，将语音信号转换为文本，从而实现语音指令的识别和响应。

(2) 回归 (Regression)

回归任务的目标是预测连续的数值输出。常见的回归算法包括：

线性回归 (Linear Regression)：通过线性方程拟合输入特征和目标值之间的关系。

在房地产领域，根据房屋的面积、房间数量、位置等特征，预测房屋的价格。线性回归模型可以建立特征与房价之间的线性关系，从而对新房屋的价格进行预测。

岭回归 (Ridge Regression)：在线性回归的基础上加入 L2 正则化项，用于防止过拟合。

在金融预测中，用于预测股票价格。由于股票价格受到多种因素（如公司业绩、市场情绪、宏观经济指标等）的影响，岭回归通过加入 L2 正则化项，可以防止模型过拟合，提高预测的稳定性。

LASSO 回归 (Least Absolute Shrinkage and Selection Operator)：在线性回归的基础上加入 L1 正则化项，可以实现特征选择。

在基因组学中，用于从大量的基因表达数据中筛选出与疾病相关的基因。LASSO 回归的 L1 正则化项可以实现特征选择，将不重要的基因特征的系数压缩为零，从而识别出关键基因。

决策树回归 (Decision Tree Regression)：通过决策树模型预测连续的数值输出。

在能源领域，用于预测电力消耗。根据天气条件（温度、湿度等）、时间（季节、小时等）等因素，决策树回归可以建立预测模型，帮助电力公司合理安排电力供应。

随机森林回归 (Random Forest Regression)：基于 Bagging 的集成学习方法，通过构建多个决策树并取平均值来提高回归性能。

在农业领域，用于预测农作物的产量。综合考虑土壤肥力、灌溉情况、气候条件等因素，随机森林回归能够提供更准确的产量预测，帮助农民优化种植策略。

支持向量回归 (Support Vector Regression, SVR)：基于 SVM 的回归算法，通过寻找最佳拟合超平面来预测连续值。

在气象学中，用于预测气温、降雨量等气象指标。SVR 通过寻找最佳拟合超平面，能够处理复杂的非线性关系，提高气象预测的准确性。

神经网络回归 (Neural Network Regression)：通过神经网络模型预测连续的数值输出，适用于复杂的回归任务。

在自动驾驶领域，用于预测车辆的行驶路径。神经网络可以学习车辆行驶过程中的各种传感器数据（如摄像头图像、激光雷达数据等），从而预测车辆在未来时刻的位置和速度。

(二) 无监督学习 (Unsupervised Learning)

无监督学习是指从未标记的数据中学习数据的内在结构。无监督学习的主要任务包括聚类和降维。

(1) 聚类 (Clustering)

聚类任务的目标是将数据划分为若干个簇，使得簇内的数据相似度高，簇间的相似度低。常见的聚类算法包括：

K-Means: 通过迭代优化簇中心和簇分配，将数据划分为 k 个簇。

在市场细分中，根据消费者的购买行为、消费偏好等特征，将消费者划分为不同的群体。例如，将消费者分为“高消费群体”、“中等消费群体”和“低消费群体”，以便企业针对不同群体制定个性化的营销策略。

高斯混合模型 (Gaussian Mixture Model, GMM): 假设数据是由多个高斯分布的混合生成的，通过 EM 算法估计每个高斯分布的参数。

在图像分割中，用于将图像中的像素划分为不同的区域（如前景和背景）。GMM 可以假设图像中的像素由多个高斯分布生成，通过估计每个高斯分布的参数，实现对图像的分割。

DBSCAN (Density-Based Spatial Clustering of Applications with Noise): 基于密度的聚类算法，通过寻找密度连通的区域来形成簇。

在地理信息系统 (GIS) 中，用于识别地理数据中的热点区域。例如，在犯罪分析中，通过 DBSCAN 可以识别犯罪活动频繁的区域，帮助警方合理分配警力。

层次聚类 (Hierarchical Clustering): 通过逐步合并或分割簇来形成层次结构，最终形成树状图 (Dendrogram)。

在生物信息学中，用于构建物种的进化树。通过计算不同物种之间的相似度，层次聚类可以逐步合并相似的物种，最终形成一个树状层次结构，展示物种进化关系。

(2) 降维 (Dimensionality Reduction)

降维任务的目标是将高维数据映射到低维空间，同时保留数据的主要特征。常见的降维算法包括：

线性判别分析 (Linear Discriminant Analysis, LDA): 通过寻找最佳投影方向，使得不同类别的数据在投影后的空间中尽可能分开。

在人脸识别中，用于提取人脸图像的特征。LDA 通过寻找最佳投影方向，使得不同人脸在投影后的空间中尽可能分开，提高人脸识别的准确性。

二、线性模型

1. 线性回归

(1) **模型定义：**线性回归是一种通过线性方程来预测目标值的方法。假设输入特征为 $x_1, x_2, \dots, x_n$ ，目标值为 $f(x)$ ，线性回归模型可以表示为 $f(x) = \theta_0 + \theta_1 x_1 + \dots + \theta_n x_n$ 。其中 $\theta_0, \theta_1, \dots, \theta_n$ 是模型参数。

(2) **代价函数：**均方误差 (MSE)， $J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$ ，衡量预测值与真实值的偏差，其中 m 是训练样本的数量， $h_{\theta}(x)$ 是模型的预测值。

优化目标是最小化代价函数

(3) **优化算法：**

梯度下降法：通过计算代价函数对每个参数的偏导数，沿着负梯度方向更新参数，逐步减小代价函数的值。

$$\text{更新公式为 } \theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta) = \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)},$$

其中学习率 α 控制步长。

解析法：通过求导令梯度为 0，直接求解 $\theta = (X^T X)^{-1} X^T y$ (适用于小规模数据)。

(4) **代码实现关键步骤：**

数据预处理 (标准化);

初始化参数;

迭代计算梯度并更新参数;

评估模型 (如均方根误差 RMSE)。

(5) **实现代码：**

Gradient Descent Algorithm

Algorithm 1: 用于线性回归的梯度下降法

Input: 样本数据集 X , y , 最大迭代次数 T

Output: w

```
1 随机初始化  $w$ 
2 for  $t = 1, \dots, T$  do
3   for  $j = 1, \dots, n$  do
4      $w_j := w_j - \alpha \frac{1}{m} \sum_{i=1}^m (f_w(x^{(i)}) - y^{(i)}) x_j^{(i)}$ 
5   end
6   if 迭代前后  $w$  变化小于  $\epsilon$  then
7     break
8   end
9 end
```

要同时更新 w_0 和 w_1

Batch: 每轮迭代都要使用所有训练样本

$$\text{Batch: } \frac{1}{m} \sum_{i=1}^m (f_w(x^{(i)}) - y^{(i)})$$

```

1 import numpy as np
2 def gradient_descent(X, y, alpha=0.01, iterations=1000):
3     m = len(y)
4     theta = np.zeros(X.shape[1])
5     for _ in range(iterations):
6         error = X @ theta - y
7         theta -= alpha * (X.T @ error) / m
8     return theta

```

(6) 训练与预测：

训练过程：通过梯度下降法等优化算法，使用训练数据来调整模型参数，使模型在训练数据上的代价函数值最小化。这个过程可能比较耗时，尤其在数据量较大时。

预测过程：在训练完成后，使用优化后的模型参数对新的输入数据进行预测。预测过程相对简单，只需要将输入数据代入线性方程即可得到预测结果。

(7) 概念模型：线性回归模型假设输入特征与目标值之间存在线性关系。模型的参数 θ 表示每个特征对目标值的影响程度。

(8) 独立同分布：在机器学习中，通常假设训练数据和测试数据是独立同分布的。这意味着训练数据和测试数据来自同一个概率分布，否则模型泛化能力下降，并且每个样本之间是独立的。例如，在围棋模型中，如果训练数据是围棋棋局，那么测试数据也应该是围棋棋局，而不是五子棋棋局。

2. Logistic 回归（分类）

(1) 模型定义：通过 Sigmoid 函数将线性组合映射到 $[0,1]$ 区间，用于二分类，如

$$h_{\theta}(x) = \frac{1}{1+e^{-\theta^T x}}.$$

(2) Sigmoid 函数： $g(z) = \frac{1}{1+e^{-z}}$ ，导数为 $g'(z) = g(z)(1 - g(z))$ 。

(3) 代价函数：交叉熵（Cross-Entropy），

$$\text{优化目标为 } \min_{\theta} J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \ln h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \ln (1 - h_{\theta}(x^{(i)})) \right],$$

$$\text{其中 } h_{\theta}(x) = \frac{1}{1+e^{-\theta^T x}}.$$

(4) 优化算法：梯度下降法，参数更新公式为 $\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$ 。

(5) 实现代码:

```
1 def sigmoid(z):
2     return 1 / (1 + np.exp(-z))
3 def logistic_regression(X, y, alpha=0.01, iterations=1000):
4     m, n = X.shape
5     theta = np.zeros(n)
6     for _ in range(iterations):
7         h = sigmoid(X @ theta)
8         theta -= alpha * (X.T @ (h - y)) / m
9     return theta
```

3. 线性判别分析 (LDA)

(1) **核心思想**: 通过投影将数据映射到低维空间(法线上), 使得同类样本在投影后的空间中尽可能接近, 不同类样本尽可能远离, 用于二分类。

(2) **代价函数思路**: 最大化类间散度与类内散度的比值 $J(w) = \frac{w^T(S_b)w}{w^T(S_w)w}$, 其中 S_b 为类间散度矩阵, S_w 为类内散度矩阵。

(3) **分割面**: 通过求解广义特征值 $S_b w = \lambda S_w w$ 问题得到最优投影方向 w , 判别式为 $f(x) = w^T x + b$ 。

4. 线性分类器和非线性分类器:

线性分类器是一种分类模型, 其决策边界是线性的。线性分类器通过学习一个线性函数来预测样本的类别。

非线性分类器是指那些能够学习到非线性决策边界的分类模型。与线性分类器不同, 非线性分类器可以处理数据中复杂的模式和关系, 适用于那些无法通过线性超平面分割的数据集。

线性分类器: logistic 回归, 支持向量机 SVM, 线性判别分析 LDA, 朴素贝叶斯, 感知器 Perceptron,

非线性分类器: 多项式核支持向量机, 高斯径向基函数核支持向量机, 决策树, 随机森林, k 最近邻 KNN, 神经网络, 深度学习模型(卷积神经网络 CNN, 循环神经网络 RNN), 贝叶斯网络, 集成学习

三、神经网络

1. 感知器 (Perceptron) 与神经元 (Neuron)

神经元模型：模拟生物神经元，输入加权求和后经激活函数输出，如 $a = g(w^T x + b)$ ，其中激活函数 g 可为 Sigmoid、ReLU 等。

感知器模型：感知器是一种简单的线性分类器，由输入层、权重、偏置和激活函数组成。它是一种单层神经网络，用于二分类任务。感知器的输出 $y = f(\sum_{i=1}^n w_i x_i + b)$ ， f 是激活函数。

感知器局限：单层感知器仅能解决线性可分问题（如与、或），无法处理异或问题。通过引入隐藏层和非线性激活函数，多层感知器可以解决非线性问题。

与 Logistic 回归的关系：若激活函数为 Sigmoid，则神经元等价于 Logistic 分类器。

2. 反向传播算法 (BP)

(一) 代价函数：常用二元交叉熵 BCE（分类）或均方误差 MSE（回归）。

基于交叉熵的损失函数：其中 $h_{\theta}(x)_k$ 表示第 k 个输出。

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K \left[y_k^{(i)} \log h_{\theta}(x^{(i)})_k + (1 - y_k^{(i)}) \log (1 - h_{\theta}(x^{(i)})_k) \right] + \frac{\lambda}{2m} \sum_{l=1}^{L-1} \sum_{i=1}^{s_l} \sum_{j=1}^{s_{l+1}} (\theta_{ji}^{(l)})^2$$

基于平方误差的损失函数：

$$E = \frac{1}{2m} \sum_{i=1}^m \sum_{k=1}^K (h_{\theta}(x^{(i)})_k - y_k^{(i)})^2$$

(二) 核心原理：通过链式求导法则计算代价函数对每个权重的梯度，将输出层误差反向传播至各层，使用梯度下降法更新权重。

(三) 计算步骤:

输入: 训练集 $D = \{(x_k, y_k)\}_{k=1}^m$;
学习率 η .

过程:

- 1: 在(0, 1)范围内随机初始化网络中所有连接权和阈值
- 2: **repeat**
- 3: **for all** $(x_k, y_k) \in D$ **do**
- 4: 根据当前参数和式(5.3) 计算当前样本的输出 $\hat{y}_k$;
- 5: 根据式(5.10) 计算输出层神经元的梯度项 g_j ;
- 6: 根据式(5.15) 计算隐层神经元的梯度项 e_h ;
- 7: 根据式(5.11)-(5.14) 更新连接权 w_{hj} , v_{ih} 与阈值 θ_j , γ_h
- 8: **end for**
- 9: **until** 达到停止条件

输出: 连接权与阈值确定的多层前馈神经网络

图 5.8 误差逆传播算法

(1) 一般:

输入训练集: $\{(x^{(1)}, y^{(1)}), \dots, (x^{(m)}, y^{(m)})\}$

初始化: 将所有 $\Delta_{ij}^{(l)} = 0$ 。

前向传播: 对每个样本 $x^{(l)}$, 计算各层激活值 $a^{(l)}$ 。 $i=1, 2, \dots, m$, $l=2, 3, \dots, L$

误差计算: 计算输出层误差 $\delta^{(L)} = a^{(L)} - y^{(l)}$, 并反向传播到隐含层, 计算

$\delta^{(L-1)}, \delta^{(L-2)}, \dots, \delta^{(2)}$ 。

梯度累积: 为每个样本累计误差, 更新 $\Delta_{ij}^{(l)} := \Delta_{ij}^{(l)} + a_j^{(l)} \delta_i^{(l+1)}$ 。

权重更新: 计算梯度并更新权重。

梯度公式:

$$\frac{\partial}{\partial \Theta_{ij}^{(l)}} J(\Theta) = D_{ij}^{(l)} = \begin{cases} \frac{1}{m} \Delta_{ij}^{(l)} + \frac{1}{m} \Theta_{ij}^{(l)} & \text{如果 } j \neq 0 \\ \frac{1}{m} \Delta_{ij}^{(l)} & \text{如果 } j = 0 \end{cases}$$

(2) 实例:

输入与前向传播:

- (a) 设置输入层的值 $a^{(1)}$ 为第 t 个训练样本 $x^{(t)}$ 。
- (b) 执行前向传播, 计算第 2 层和第 3 层的激活值 $a^{(2)}$ 和 $a^{(3)}$ 。

输出层误差计算:

- (c) 对于输出层的每个单元 k , 设置: $\delta_k^{(3)} = a_k^{(3)} - y_k$

面向隐含层的误差反向传播:

- (d) 对于隐藏层 $l = 2$, 设置: $\delta^{(2)} = (\theta^{(2)})^T \delta^{(3)} \odot g'(z^{(2)})$

使用误差梯度计算模型参数梯度:

- (e) 累积梯度: $\Delta^{(l)} = \Delta^{(l)} + \delta^{(l+1)} (a^{(l)})^T$
- (f) 计算未正则化的梯度: $\frac{\partial J(\theta)}{\partial \theta^{(l)}} = \frac{1}{m} \Delta^{(l)}$

使用梯度下降法或梯度共轭法优化模型参数:

- (g) 更新所有连接权重: $\theta^{(l)} = \theta^{(l)} - \alpha \frac{\partial J(\theta)}{\partial \theta^{(l)}}$

(3) 示例:

基于交叉熵

梯度的计算(基于交叉熵)

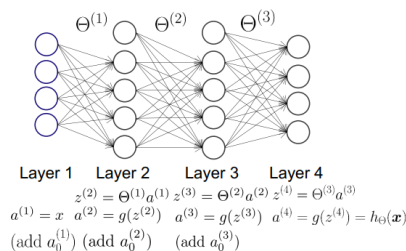


$$J(\Theta) = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K \left(y_k^{(i)} \log(h_{\Theta}(x^{(i)}))_k + (1 - y_k^{(i)}) \log((1 - h_{\Theta}(x^{(i)}))_k) \right) + \frac{\lambda}{2m} \sum_{l=1}^{L-1} \sum_{i=1}^{s_l} \sum_{j=1}^{s_{l+1}} (\Theta_{ij}^{(l)})^2$$

$$\min_{\Theta} J(\Theta)$$

Need code to compute:

$$-J(\Theta) \\ -\frac{\partial}{\partial \Theta_{ij}^{(l)}} J(\Theta)$$



梯度计算：基于误差的梯度计算



对于一个样本和一个输出层节点的误差相对于模型参数的梯度：

$$\frac{\partial J}{\partial \Theta^{(2)}} = \frac{\partial J}{\partial a^{(4)}} \frac{\partial a^{(4)}}{\partial z^{(4)}} \cdot \underbrace{\frac{\partial z^{(4)}}{\partial a^{(3)}} \frac{\partial a^{(3)}}{\partial z^{(3)}}}_{\delta^{(3)}} \cdot \frac{\partial z^{(3)}}{\partial \Theta^{(2)}}$$

$$\Theta_{ij}^{(2)}(t+1) = \Theta_{ij}^{(2)}(t) - \alpha \times \frac{\partial J}{\partial \Theta_{ij}^{(2)}}$$

梯度计算：误差(可复用)、误差的反向传递



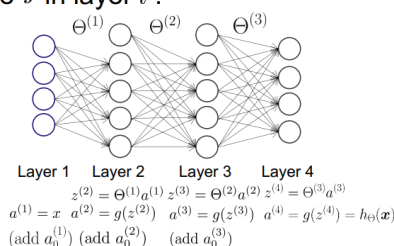
Intuition: $\delta_j^{(l)}$ = “error” of node j in layer l .

损失函数相对于模型参数(网络连接权重)的梯度使用链式求导法计算

$$\delta^{(4)} = \frac{\partial J}{\partial a^{(4)}} \frac{\partial a^{(4)}}{\partial z^{(4)}}$$

$$\delta^{(3)} = \frac{\partial J}{\partial a^{(4)}} \frac{\partial a^{(4)}}{\partial z^{(4)}} \frac{\partial z^{(4)}}{\partial a^{(3)}} \frac{\partial a^{(3)}}{\partial z^{(3)}} = \underbrace{\delta^{(4)}}_{\delta^{(4)}} \frac{\partial z^{(4)}}{\partial a^{(3)}} \frac{\partial a^{(3)}}{\partial z^{(3)}}$$

$$\delta^{(2)} = \frac{\partial J}{\partial a^{(4)}} \frac{\partial a^{(4)}}{\partial z^{(4)}} \cdot \underbrace{\frac{\partial z^{(4)}}{\partial a^{(3)}} \frac{\partial a^{(3)}}{\partial z^{(3)}}}_{\delta^{(3)}} \cdot \frac{\partial z^{(3)}}{\partial a^{(2)}} \frac{\partial a^{(2)}}{\partial z^{(2)}}$$



BackPropagation算法的关键计算：



$$\delta^{(4)} = \frac{\partial J}{\partial a^{(4)}} \frac{\partial a^{(4)}}{\partial z^{(4)}} = a^{(4)} - y$$

$$\frac{\partial J}{\partial a^{(4)}} = \frac{a^{(4)} - y^{(i)}}{a^{(4)}(1 - a^{(4)})} \quad \text{求导见下一页}$$

$$\frac{\partial a^{(4)}}{\partial z^{(4)}} = a^{(4)}(1 - a^{(4)}) \quad \text{求导见下下页}$$

GOAL: $\delta^{(4)} = \frac{\partial J}{\partial a^{(4)}} \frac{\partial a^{(4)}}{\partial z^{(4)}} = a^{(4)} - y$



$$\begin{aligned} \frac{\partial J}{\partial a^{(4)}} &= \frac{\partial}{\partial a^{(4)}} \left[- \left(y^{(i)} \log a^{(4)} + (1 - y^{(i)}) \log(1 - a^{(4)}) \right) \right] \\ &= -y^{(i)} \frac{1}{a^{(4)}} - (1 - y^{(i)}) \cdot \frac{1}{1 - a^{(4)}} \cdot (-1) \\ &= -\frac{y^{(i)}}{a^{(4)}} + \frac{1 - y^{(i)}}{1 - a^{(4)}} \\ &= \frac{-y^{(i)} + y^{(i)} a^{(4)} + a^{(4)} - a^{(4)} y^{(i)}}{a^{(4)}(1 - a^{(4)})} \\ &= \frac{a^{(4)} - y^{(i)}}{a^{(4)}(1 - a^{(4)})} \end{aligned}$$

GOAL: $\delta^{(4)} = \frac{\partial J}{\partial a^{(4)}} \frac{\partial a^{(4)}}{\partial z^{(4)}} = a^{(4)} - y$



$$\begin{aligned}\frac{\partial a^{(4)}}{\partial z^{(4)}} &= \frac{\partial}{\partial z^{(4)}} \left(\frac{1}{1 + e^{-z^{(4)}}} \right) \\ &= \frac{-1}{(1 + e^{-z^{(4)}})^2} \cdot \frac{\partial}{\partial z^{(4)}} (e^{-z^{(4)}}) \\ &= \frac{-1}{(1 + e^{-z^{(4)}})^2} \cdot e^{-z^{(4)}} \cdot (-1) \\ &= \frac{1}{1 + e^{-z^{(4)}}} \cdot \frac{e^{-z^{(4)}}}{1 + e^{-z^{(4)}}} \\ &= a^{(4)}(1 - a^{(4)})\end{aligned}$$

对于sigmoid函数

$$g(z) = \frac{1}{1 + e^{-z}}$$

其导数为:

$$g'(z) = g(z)(1 - g(z))$$

汇总整理

$$\frac{\partial J}{\partial \Theta^{(2)}} = \frac{\partial J}{\partial a^{(4)}} \frac{\partial a^{(4)}}{\partial z^{(4)}} \cdot \frac{\partial z^{(4)}}{\partial a^{(3)}} \frac{\partial a^{(3)}}{\partial z^{(3)}} \cdot \frac{\partial z^{(3)}}{\partial \Theta^{(2)}}$$



$$\delta^{(4)} = \frac{\partial J}{\partial a^{(4)}} \frac{\partial a^{(4)}}{\partial z^{(4)}} = a^{(4)} - y$$

$$\frac{\partial J}{\partial \Theta^{(3)}} = \delta^{(4)} \cdot \frac{\partial z^{(4)}}{\partial \Theta^{(3)}} = \delta^{(4)} \cdot a^{(3)}$$

$$\begin{aligned}\delta^{(3)} &= \frac{\partial J}{\partial a^{(4)}} \frac{\partial a^{(4)}}{\partial z^{(4)}} \frac{\partial z^{(4)}}{\partial a^{(3)}} \frac{\partial a^{(3)}}{\partial z^{(3)}} \\ &= (\Theta^{(3)})^T \times \delta^{(4)} \cdot *g'(z^{(3)})\end{aligned}$$

$$\frac{\partial J}{\partial \Theta^{(2)}} = \delta^{(3)} \cdot \frac{\partial z^{(3)}}{\partial \Theta^{(2)}} = \delta^{(3)} \cdot a^{(2)}$$

$$\begin{aligned}\delta^{(2)} &= \frac{\partial J}{\partial a^{(4)}} \frac{\partial a^{(4)}}{\partial z^{(4)}} \cdot \frac{\partial z^{(4)}}{\partial a^{(3)}} \frac{\partial a^{(3)}}{\partial z^{(3)}} \cdot \frac{\partial z^{(3)}}{\partial a^{(2)}} \frac{\partial a^{(2)}}{\partial z^{(2)}} \\ &= (\Theta^{(2)})^T \times \delta^{(3)} \cdot *g'(z^{(2)})\end{aligned}$$

$$\frac{\partial J}{\partial \Theta^{(1)}} = \delta^{(2)} \cdot \frac{\partial z^{(2)}}{\partial \Theta^{(1)}} = \delta^{(2)} \cdot a^{(1)}$$

基于均方差

$$E = \frac{1}{2} \sum_{i=1}^m (a^{(4)} - y^{(i)})^2$$

$$\delta^{(4)} = \frac{\partial E}{\partial a^{(4)}} \cdot \frac{\partial a^{(4)}}{\partial z^{(4)}} = (a^{(4)} - y) \cdot a^{(4)}(1 - a^{(4)})$$

$$\frac{\partial E}{\partial \theta_{ij}^{(3)}} = \delta^{(4)} \frac{\partial z^{(4)}}{\partial \theta_{ij}^{(3)}} = \delta^{(4)} \cdot a^{(3)}$$

其余各层同交叉熵表示形式一样

四、深度学习

深度学习是机器学习的一个分支，其基于一系列算法，试图对数据中的高层次抽象进行建模。深度学习的优势在于能够自动提取特征，而非手动提取特征。

(一) 主要任务

计算机视觉 (CV)：图像分类、目标检测、图像分割。

例如，使用卷积神经网络 (CNN) 对图像进行分类，识别图像中的物体类别。

自然语言处理 (NLP)：机器翻译、文本生成 (GPT 系列)、情感分析。

例如，使用 Transformer 架构进行机器翻译，将一种语言的文本翻译成另一种语言。

深度信念网络 (DBN)：深度信念网络是一种生成模型，由多个受限玻尔兹曼机 (RBM) 堆叠而成。

DBN 在语音识别等领域取得了突破，引起了人们对深度学习的关注。

(二) 卷积神经网络 (CNN)

卷积神经网络是一种前馈神经网络，具有结构简单、训练参数少和适应性强的特点。CNN 避免了图像等复杂的预处理（如人工提取特征），可以直接输入原始图像。

基本组成部分：

卷积层：卷积层是 CNN 的核心层，通过卷积核在输入图像上滑动，提取图像的局部特征。卷积操作可以保留图像的空间信息，同时减少参数数量。

池化层：池化层用于对卷积层的输出进行降采样，减少特征图的尺寸，降低计算量和过拟合风险（降维并保留关键特征）。常见的池化操作包括最大池化和平均池化。

Softmax 层：将 CNN 的输出映射到概率分布，表示样本属于每个类别的概率。

激活函数：ReLU ($f(x) = \max(0, x)$)，解决梯度消失问题，加速神经网络的训练。。

(三) MindSpore 框架

MindSpore 是华为推出的一种深度学习框架，支持端、边、云的协同训练和推理。它提供了丰富的 API 和工具，方便开发者构建和优化深度学习模型。

作用：MindSpore 的主要作用是提供一个高效、灵活的深度学习开发平台，支持多种硬件架构，包括昇腾芯片、GPU 等。它可以帮助开发者快速实现深度学习模型的训练和部署。

结构：

1. 硬件层（Hardware Layer）

(1) 计算载体

云（Cloud）：基于昇腾 AI 芯片的服务器集群，提供大规模的计算资源

边（Edge）：边缘设备，用于处理靠近数据源的数据

端（Device）如智能手机、平板电脑等移动设备，运行轻量化模型

(2) 芯片与板卡

昇腾（Ascend）AI 芯片

Ascend 310/910：华为自研的 AI 处理器，专为 AI 训练和推理设计，提供高性能和高能效比

兼容硬件

通过 CANN 支持其他异构硬件（如 GPU/CPU）的协同计算

2. 驱动层（Driver Layer）

CANN（Compute Architecture for Neural Networks）

华为自研的统一异构计算架构，提供对多种硬件的驱动支持和优化，支持昇腾/GPU/CPU 混合计算。

CUDA（兼容层）

通过适配层支持 NVIDIA GPU 的 CUDA 生态，实现跨平台兼容性。

3. 深度计算平台框架（MindSpore Ecosystem）

(1) 开发工具链

Mind Studio

集成开发环境（IDE），支持模型开发、调试和性能调优

Model Zoo

提供预训练模型库，用户可以直接使用或基于这些模型进行二次开发。

(2) 数据处理与增强

Mind Data

数据加载、格式转换和数据增强工具，帮助用户高效处理数据

(3) 安全与隐私

Mind Armour

安全工具，提供模型混淆、端云协同隐私保护等功能

(4) 性能优化工具

Mind Insight

调试调优工具，提供模型训练过程中的可视化和分析功能

Mind Compiler

提供编译优化功能，提高模型运行效率。包含：类型推导、自动微分、内存优化、图算融合等

MindIR：中间表示，用于模型的优化和转换

MindAKG (poly 自动优化)：自动优化工具，进一步提升模型性能

(5) 运行时系统

Mind RT：

运行时环境，主要负责模型的部署和推理执行，支持分布式异构并行，以实现高效的模型运行。

(6) 前端支持

仓颉/Julia：多语言前端接口，降低开发者门槛。

(7) 扩展库 (MindSpore Extend)

领域模型库

GNN：图神经网络工具包

大语言模型：支持 Transformer 类模型训练与推理

强化学习：提供环境接口和算法实现

科学计算：微分方程求解器等

MindX 系列

MindX SDK：行业 SDK，提供特定行业场景的解决方案

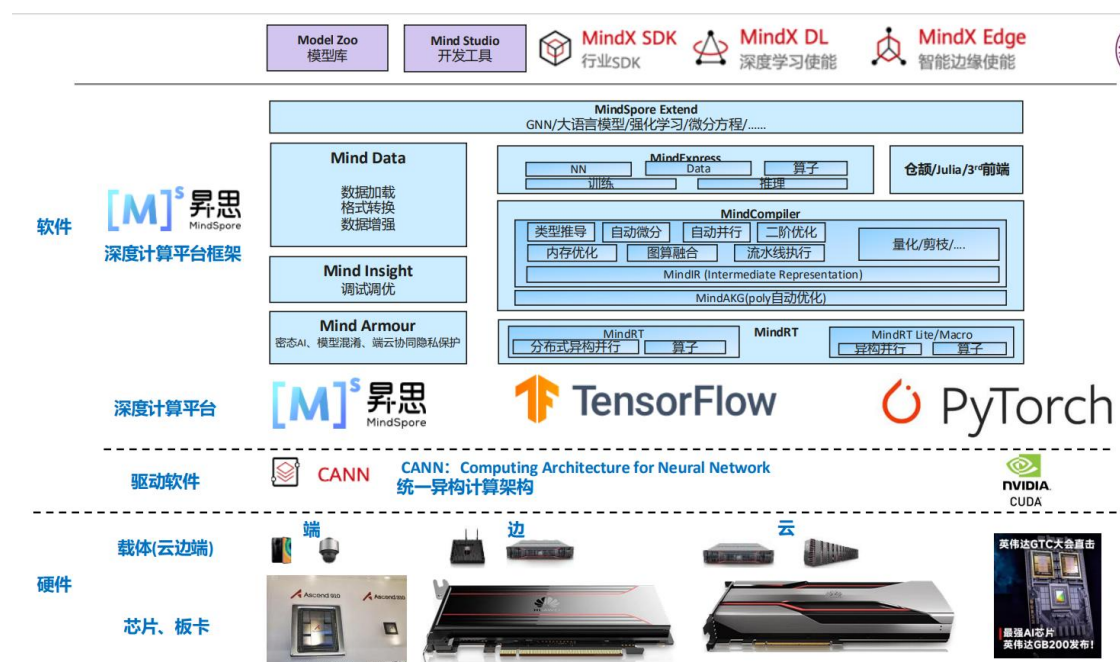
MindX DL：深度学习使能，提供深度学习模型开发和训练的工具和库

MindX Edge：智能边缘使能，支持在边缘设备上部署和运行 AI 模型

4. 深度计算平台 (Platform Integration)

昇思 MindSpore, TensorFlow, PyTorch

5. 结构图



(四) 与 PyTorch/TensorFlow 的对比：

支持动态图和静态图，侧重国产硬件适配，支持自动微分和并行计算。

PyTorch: PyTorch 是一种动态图深度学习框架，支持动态计算图和自动微分。它在研究和开发中非常灵活，适合快速原型开发。

TensorFlow: TensorFlow 是一种静态图深度学习框架，支持大规模分布式训练和部署。它在工业界和学术界都有广泛的应用。

(五) 深度学习训练中的常见技巧

Data Augmentation (数据增强): 对训练数据进行如翻转、裁剪、缩放等变换，增加数据多样性，提升模型泛化能力。

Dropout (随机失活): 训练时随机让部分神经元不工作，减少过拟合，增强模型鲁棒性。

ReLU (修正线性单元): 一种非线性激活函数，缓解梯度消失，加快训练速度。

Batch Normalization (批量归一化): 对每个小批量数据的特征做归一化，加速训练收敛，提升模型稳定性。

(六) 名词解释

GPT: 生成式预训练变换器 (Generative Pre-trained Transformer, 简称 GPT) 是一种先进的人工智能语言模型, 它通过深度学习技术, 特别是 Transformer 架构理解和生成自然语言文本。GPT 通过在大量文本数据上的预训练[1], 学习语言的模式和结构, 使其能够预测和生成连贯、有意义的文本内容。GPT 模型可以广泛应用于文本生成、对话系统、自动摘要等多种自然语言处理任务。

LeNet: LeNet 是杨立昆于 1998 年为手写数字识别设计的卷积神经网络。它是早期卷积神经网络中最具代表性的实验系统之一。LeNet 包含卷积层、池化层和全连接层, 这些都是现代 CNN 网络的基本组成部分。LeNet 被认为是 CNN 的开端。网络结构: 3 个卷积层 + 2 个池化层 + 1 个全连接层 + 1 个输出层

图卷积神经网络 GCN/GNN: 图卷积神经网络是一种用于处理图结构数据的深度学习模型。它通过考虑图中节点的邻居信息来更新节点的特征, 从而学习节点的表示。这种方法特别适用于社交网络、推荐系统、生物信息学等领域, 因为它能够有效地捕捉节点间的复杂关系。GCN 通过图卷积层来实现这一过程, 每一层都会聚合邻居节点的特征, 并更新当前节点的特征。

五、支持向量机 (SVM)

1. 线性可分 SVM (最大间隔分类)

最大间隔：线性可分 SVM 的目标是找到一个超平面，使得两类样本之间的间隔最大化。间隔为 $\frac{2}{|w|}$ ，是指超平面到最近样本点的距离，最大间隔超平面可以提高模型的泛化能力。

代价函数： $\min_{w,b} \frac{1}{2} |w|^2$ ，约束条件为 $y_i(w^T x_i + b) \geq 1$ 。其中 w 是超平面的法向量。通过最小化这个代价函数，可以找到最大间隔超平面。

拉格朗日乘子法：一种优化方法，将约束问题转化为无约束问题，对于线性可分 SVM，拉格朗日函数为 $L(w, b, \alpha) = \frac{1}{2} |w|^2 + \sum_{i=1}^m \alpha_i [1 - y_i(w^T x_i + b)]$ ，其中 α_i 是拉格朗日乘子。

最大最小对偶：通过拉格朗日乘子法，可以将原问题转化为对偶问题，其形式为 $\max_{\alpha} \left\{ \min_{w,b} L(w, b, \alpha) \right\}$ 。对偶问题是 $\max_{\alpha} \left(\sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j x_i^T x_j \right)$ ，约束 $\sum_{i=1}^m \alpha_i y_i = 0$ ， $\alpha_i \geq 0$ 。通过求解对偶问题，可以得到最优的 α_i ，进而求得 w 和 b 。

- **不等式约束 (KKT条件)：**

对于问题 $\min f(x)$ s.t. $g(x) \leq 0$ ，拉格朗日函数为：

$$\mathcal{L}(x, \lambda) = f(x) + \lambda g(x)$$

需满足以下KKT条件：

$$\begin{cases} \nabla_x \mathcal{L} = 0 \\ g(x) \leq 0 \\ \lambda \geq 0 \\ \lambda g(x) = 0 \end{cases}$$

(最后一条称为互补松弛条件)

- **多约束问题：**

若有多个约束 $g_i(x) = 0$ 和 $h_j(x) \leq 0$ ，拉格朗日函数为：

$$\mathcal{L}(x, \lambda, \mu) = f(x) + \sum_i \lambda_i g_i(x) + \sum_j \mu_j h_j(x)$$

其中 $\mu_j \geq 0$ 。

3. 求解步骤

1. 构造拉格朗日函数：

$$\mathcal{L}(x, \lambda) = f(x) + \lambda g(x)。$$

2. 对变量和乘子求偏导：

◦ 对 x 求梯度： $\nabla_x \mathcal{L} = \nabla f(x) + \lambda \nabla g(x) = 0$ 。

◦ 对 λ 求导： $\frac{\partial \mathcal{L}}{\partial \lambda} = g(x) = 0$ (即原始约束)。

3. 解方程组：

联立方程 $\nabla_x \mathcal{L} = 0$ 和 $g(x) = 0$ ，求出 x 和 λ 。

支持向量：满足 $y_i(w^T x_i + b) = 1$ 的样本，决定分类超平面

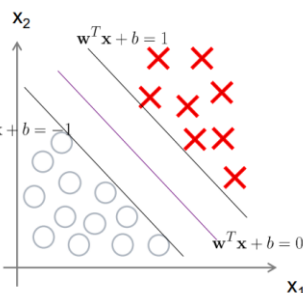
当使用训练数据集优化代价函数

$$L(\mathbf{w}, b, \alpha) = \frac{1}{2} \|\mathbf{w}\|^2 + \sum_{i=1}^m \alpha_i (1 - y_i(\mathbf{w}^T \mathbf{x}_i + b))$$

得到模型参数 $\mathbf{w}, b$ 后, 判决函数

为:

$$f(\mathbf{x}_i; \mathbf{w}) = \begin{cases} 1, & \mathbf{w}^T \mathbf{x}_i + b \geq 1 \\ -1, & \mathbf{w}^T \mathbf{x}_i + b \leq -1 \end{cases}$$



2. 软间隔 SVM (CSVM)

松弛因子: 在实际应用中, 数据往往是线性不可分的, 或者存在噪声。软间隔 SVM 通过引入松弛因子 ξ_i 来允许一些样本违反约束条件, 从而提高模型的鲁棒性。

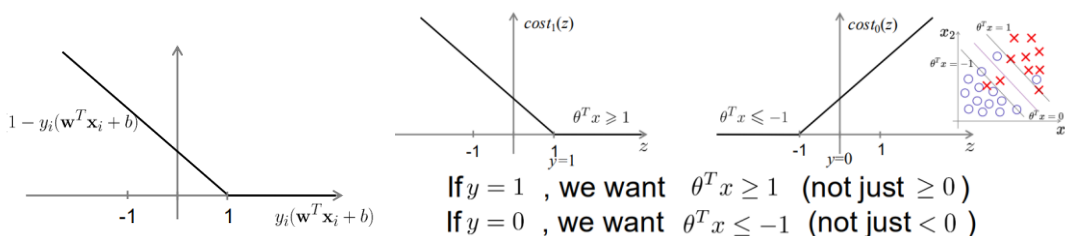
代价函数: $\min_{\mathbf{w}, b, \xi_i} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m \xi_i$, $y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i$, $\xi_i \geq 0$ 。包括两部分: 一部

分是间隔最大化项 $\frac{1}{2} \|\mathbf{w}\|^2$, 另一部分是松弛因子的惩罚项 $C \sum_{i=1}^m \xi_i$ 。其中 C 是惩罚参数, 用于控制松弛因子的权重, C 越大越严格。通过最小化这个代价函数, 可以找到最优的超平面。

拉格朗日函数: $L(\mathbf{w}, b, \alpha, \xi, \mu) = \frac{1}{2} \|\mathbf{w}\|^2 + \sum_{i=1}^m \alpha_i [1 - \xi_i - y_i(\mathbf{w}^T \mathbf{x}_i + b)] + C \sum_{i=1}^m \xi_i - \sum_{i=1}^m \mu_i \xi_i$

对偶: $\max_{\alpha} \left\{ \min_{\mathbf{w}, b, \xi, \mu} L(\mathbf{w}, b, \alpha, \xi, \mu) \right\} = \max_{\alpha} \left(\sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j \right)$,
 $\sum_{i=1}^m \alpha_i y_i = 0$, $0 \leq \alpha_i \leq C$

折页损失 (Hinge Loss): $L(y, f(x)) = \max(0, 1 - yf(x))$, 衡量分类错误的代价, 当预测正确且间隔足够大时损失为 0。



左为折页, 右为吴恩达

图像中的表示: 吴恩达课件中常用 $y \in \{0, 1\}$, 标准 SVM 中 $y \in \{-1, 1\}$, 表示正类和负类。在软间隔 SVM 中, 超平面的方程为 $\mathbf{w}^T \mathbf{x} + b = 0$, 正类和负类的样本分别位于超平面的两侧。

3. 核技术

目的：将低维非线性数据映射到高维空间，使其线性可分。

常用核函数：

核函数的计算不需要显式地将数据映射到高维空间，而是通过计算输入数据之间的内积来实现

线性核： $K(x_i, x_j) = x_i^T x_j$

多项式核： $K(x_i, x_j) = (x_i^T x_j)^d$

高斯核 (RBF)： $K(x_i, x_j) = \exp\left(-\frac{|x_i - x_j|^2}{2\sigma^2}\right)$ 。

六、机器学习模型评估

1. 评估指标

全面性：评估模型性能时，需要综合考虑多个指标，以全面了解模型的性能。常见的评估指标包括准确率、召回率、F1 分数、ROC 曲线、AUC 等。

(1) 分类任务：

准确率 (Accuracy)：表示模型正确分类的样本比例。准确率的计算公式为 $Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$ ，其中 TP 表示真正例，TN 表示真负例，FP 表示假正例，FN 表示假负例

查准率 (Precision)：即在分类器预测出的正例中，真正正确的个数占整个结果的比
例， $Precision = \frac{TP}{TP+FP}$

查全率 (Recall)：表示模型正确识别的正例比例。查全率的计算公式为 $Recall = \frac{TP}{TP+FN}$

F1 分数：是查准率和查全率的调和平均数，用于综合考虑查准率和查全率。F1 分数的计算公式为 $F1 = \frac{Precision \cdot Recall}{Precision + Recall}$

ROC 曲线 (真正率 vs 假正率)：用于评估模型在不同阈值下的性能。ROC 曲线以假正例率 ($FPR = \frac{FP}{FP+TN}$) 为横轴，真正例率 ($TPR = \frac{TP}{TP+FN}$) 为纵轴

AUC (曲线下面积)：表示 ROC 曲线下的面积，AUC 越大，模型的性能越好。

(2) 回归任务：均方误差 (MSE)、平均绝对误差 (MAE)。

(3) 代价敏感指标：在某些应用中，不同类型的错误可能有不同的代价。

例如，在医疗诊断中，将患者误诊为健康人可能比将健康人误诊为患者更严重。代价敏感指标可以考虑这些不同代价，对模型进行更合理的评估。

2. 评估环境与方法

(1) 交叉验证：

交叉验证是一种常用的模型评估方法，通过将数据集划分为多个子集，分别用于训练和验证，从而评估模型的泛化能力。常见的交叉验证方法包括 k 折交叉验证、留一交叉验证等。

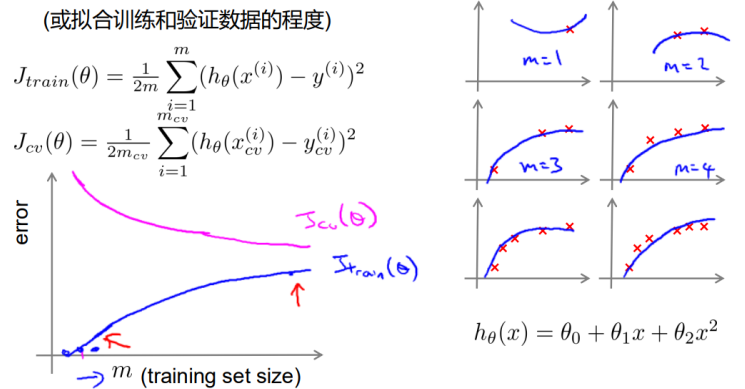
K 折交叉验证：将数据分为 K 份，每次用 K-1 份训练，1 份测试，重复 K 次。

留一交叉验证 (LOOCV): 每次只留一个样本作为验证集, 其余样本作为训练集。这种方法可以充分利用数据, 但计算量较大, 适用于小数据集。

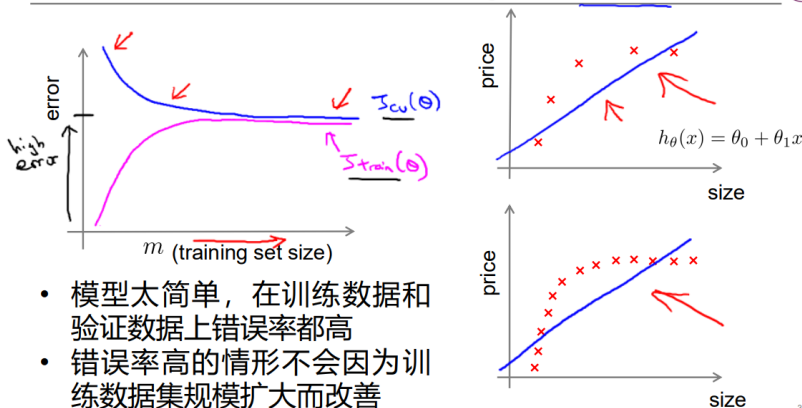
(2) 训练误差与测试误差: 训练误差表示模型在训练数据上的误差, 测试误差表示模型在测试数据上的误差。通过比较训练误差和测试误差, 可以评估模型的过拟合或欠拟合情况。理想的模型应该在训练数据和测试数据上都有较低的误差。

(3) 学习曲线

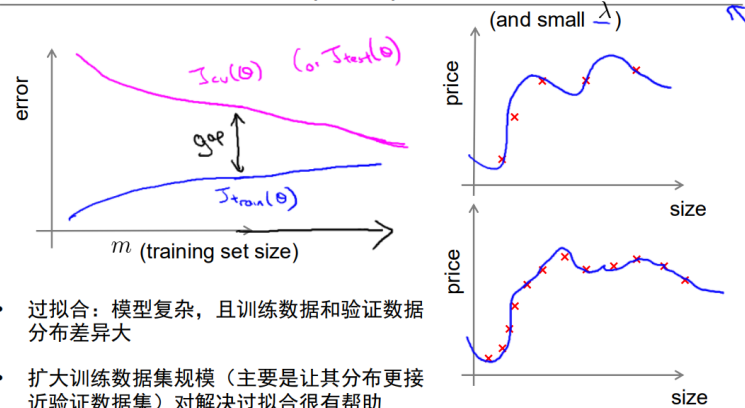
3.3 使用学习曲线(Learning curves)来反映模型的学习程度



3.3 使用学习曲线展示欠拟合(Underfit, High bias)



3.3 使用学习曲线展示过拟合(高方差)



七、贝叶斯分类器（无大题）

贝叶斯是生成式模型（对比于判决式模型），是概率算法（对比于统计算法）

1. 朴素贝叶斯

独立性假设：朴素贝叶斯分类器假设各个特征之间相互独立。在计算联合概率时，可以将各个特征的概率相乘。例如，对于一个样本 $x = (x_1, x_2, \dots, x_n)$ ，朴素贝叶斯分类器计算联合概率为 $P(x|y) = \prod_{i=1}^n P(x_i|y)$ 。

联合概率：朴素贝叶斯分类器通过计算每个类别的联合概率 $P(y|x) = P(y) \prod_{i=1}^n P(x_i|y)$ ，选择联合概率最大的类别作为预测结果。其中 $P(y)$ 是先验概率，表示类别 y 出现的概率； $P(x_i|y)$ 是条件概率，表示在类别 y 下特征 x_i 出现的概率。

分类公式（判决函数）： $h_{nb} = \arg \max_c P(c) \prod_{i=1}^d P(x_i|c)$

类先验概率： $P(c) = \frac{|D_c|}{|D|}$ 类条件概率（离散）： $P(x_i|c) = \frac{|D_{c,x_i}|}{|D_c|}$

平滑处理：在实际应用中，可能会遇到某些特征在某个类别下从未出现的情况，导致条件概率为零。为了避免这种情况，可以使用平滑处理，如拉普拉斯平滑。

$\hat{P}(c) = \frac{|D_c|+1}{|D|+N}$ ， N ：数据集中的类别数

$\hat{P}(x_i|c) = \frac{|D_{c,x_i}|+1}{|D_c|+N_i}$ ， N_i ：第 i 维特征的可能取值数

2. 贝叶斯信念网（不重要）

贝叶斯信念网：贝叶斯信念网是一种有向无环图，用于表示变量之间的概率依赖关系。每个节点表示一个变量，边表示变量之间的依赖关系。通过贝叶斯信念网，可以计算联合概率分布，并进行推理和决策。

条件概率表（CPT）：贝叶斯信念网中每个节点都有一个条件概率表，表示在父节点取不同值的情况下，该节点的概率分布。例如，对于一个节点 X ，其父节点为 Y 和 Z ，条件概率表表示为 $P(X|Y,Z)$ 。

3. 最大似然估计（MLE）

思想：最大似然估计是一种参数估计方法，假设数据服从某分布，如高斯分布的均值和方差。通过最大化似然函数估计分布参数，似然函数表示在给定参数的情况下，观测数据出现的概率。

步骤：构造似然函数 $L(\theta) = \prod_{i=1}^m P(x_i|\theta)$ ，取对数后求导找极值点。

5. 极大似然估计

• 以估计类条件概率分布为例

令 D_c 表示训练集 D 中第 c 类样本组成的集合，假设这些样本是独立同分布的，则参数 θ_c 对于数据集 D_c 的似然是

$$P(D_c | \theta_c) = \prod_{\mathbf{x} \in D_c} P(\mathbf{x} | \theta_c). \quad (7.9)$$

对 θ_c 进行极大似然估计，就是去寻找能最大化似然 $P(D_c | \theta_c)$ 的参数值 $\hat{\theta}_c$ 。直观上看，极大似然估计是试图在 θ_c 所有可能的取值中，找到一个能使数据出现的“可能性”最大的值。

5.Log-likelihood: 对似然进行对数化

式(7.9)中的连乘操作易造成下溢，通常使用对数似然(log-likelihood)

$$\begin{aligned} LL(\theta_c) &= \log P(D_c | \theta_c) \\ &= \sum_{\mathbf{x} \in D_c} \log P(\mathbf{x} | \theta_c), \end{aligned} \quad (7.10)$$

此时参数 θ_c 的极大似然估计 $\hat{\theta}_c$ 为

$$\hat{\theta}_c = \arg \max_{\theta_c} LL(\theta_c). \quad (7.11)$$

4. 几个独立

训练和预测：训练数据集和测试数据集独立同分布

朴素贝叶斯：特征独立

最大似然：训练样本从训练数据中独立抽样出来

八、网络机器学习

1. PageRank 算法

核心思想：PageRank 是 Google 搜索引擎中用于网页排序的核心算法。它通过模拟互联网用户的随机游走行为，为每个网页分配一个权威度值，表示网页的重要性。

PageRank 算法的基本思想是：一个网页的权威度不仅取决于指向它的链接数量，还取决于这些链接来源网页的权威度。

随机游走模型：迭代公式为 $PR(p_i) = d \sum_{p_j \in M(p_i)} \frac{PR(p_j)}{L(p_j)} + \frac{1-d}{N}$ ，其中 N 是节点个数（网页总数）， $M(p_i)$ 是链入 p_i 的页面集合， $L(p_j)$ 是 p_j 链出的页面，d 是继续访问下一个页面的概率，1-d 是停止点击并随机浏览新页面的概率

应用：网页排序、SCI 期刊影响因子评估、社交网络分析、学术论文引用分析等。

2. 带重启的随机游走（RWR）

思想：带重启的随机游走是一种改进的随机游走算法，它在随机游走过程中引入了重启机制。在每次跳转时，除了按照链接概率跳转到其他网页外，还以一定概率返回到初始网页。这种重启机制可以防止随机游走陷入局部区域，提高算法的全局性。

周登勇算法：结合局部和全局一致性，损失函数为

$$Q(F) = \frac{1}{2} \left(\sum_{i,j=1}^n W_{i,j} \left| \frac{F_i}{\sqrt{D_{ii}}} - \frac{F_j}{\sqrt{D_{jj}}} \right|^2 + \mu \sum_{i=1}^n |F_i - Y_i|^2 \right), \text{ 分类函数 } F^* = \arg \min_{F \in \mathcal{F}} Q(F)$$

九、集成学习

1. Boosting

核心思想：通过组合多个弱学习器来构建一个强学习器。迭代训练弱学习器，逐步调整样本权重，使得每次训练的弱学习器能够更关注之前学习器错误分类的样本。最终预测由所有弱学习器的加权投票决定。

AdaBoost 算法步骤：

Algorithm 1: AdaBoost.M1

Input: 训练数据集 D , 子分类器学习算法, 迭代次数 T

Output: $F(x) = \sum_{t=1}^T \alpha_t f_t(x)$

1 初始化各训练样本权重 $w_1^{(i)} = \frac{1}{m}$

2 **for** $t = 1, \dots, T$ **do**

3 使用带权重 $w_t^{(i)}$ 的训练数据集 D 训练得到子分类器 $f_t(x)$

4 计算 $f_t(x)$ 的权重: $\alpha_t = \frac{1}{2} \ln \frac{1-\epsilon_t}{\epsilon_t}$, 其中, 错误率 $\epsilon_t = \frac{\sum_{i=1}^m w_i I(y^{(i)} \neq f_t(x^{(i)}))}{\sum_{i=1}^m w_i}$ 。当

$\epsilon > 0.5$ 时 $\alpha_t = 0$

5 更新样本权重: $w_{t+1}^{(i)} = \frac{w_t^{(i)}}{Z_t} \exp(-y^{(i)} \alpha_t f_t(x^{(i)}))$, 其中,
 $Z_t = \sum_{i=1}^m w_t^{(i)} \exp(-y^{(i)} \alpha_t f_t(x^{(i)}))$

6 **end**

最终模型 $F_T(x) = \sum_{t=1}^T \alpha_t f_t(x)$

指数损失函数：

整个集成模型： $L(F_t(x)) = \sum_{i=1}^m e^{-y^{(i)} F_t(x^{(i)})}$

单个弱分类器： $L(\alpha_t f_t(x) | w_t) = \sum_{i=1}^m w_t^{(i)} e^{-y^{(i)} \alpha_t f_t(x^{(i)})}$, 其中 $w_t^{(i)} = e^{-y^{(i)} F_{t-1}(x^{(i)})}$

应用：AdaBoost 在分类任务中表现出色，尤其适用于二分类问题。

2. Bagging

核心思想：通过自助采样 (Bootstrap Sampling) 生成多个训练集，每个训练集用于训练一个基学习器，最终通过多数投票或平均值来得到最终结果。

随机森林：以决策树为基学习器，引入随机属性选择，降低方差。先从 d 个属性中随机选择 k 个属性，再从这 k 个属性中选择一个最优属性

3. Stacking

用次级学习器融合初级学习器的输出，如初级学习器输出作为次级学习器的输入。

十、聚类（无大题）

（无监督学习）

1. K-Means 算法

思想：K-Means 是一种基于划分的聚类算法，通过将数据集划分为 k 个簇，使得簇内相似度最大化，簇间相似度最小化。K-Means 的核心思想是通过迭代优化簇中心和簇分配，逐步减小平方误差。

步骤：

- (1) 随机初始化 K 个簇中心 $\mu_1, \mu_2, \dots, \mu_k$
- (2) 对于每个样本 $x^{(i)}$ ，计算其与每个簇中心的距离，将其分配到最近的簇中心
- (3) 更新每个簇的中心为该簇所有样本的均值
- (4) 重复步骤(2)和(3)，直到簇中心不再变化或达到最大迭代次数

输入： 样本集 $D = \{x_1, x_2, \dots, x_m\}$;
聚类簇数 k .

过程：

- 1: 从 D 中随机选择 k 个样本作为初始均值向量 $\{\mu_1, \mu_2, \dots, \mu_k\}$
- 2: **repeat**
- 3: 令 $C_i = \emptyset$ ($1 \leq i \leq k$)
- 4: **for** $j = 1, 2, \dots, m$ **do**
- 5: 计算样本 x_j 与各均值向量 μ_i ($1 \leq i \leq k$) 的距离: $d_{ji} = \|x_j - \mu_i\|_2$;
- 6: 根据距离最近的均值向量确定 x_j 的簇标记: $\lambda_j = \arg \min_{i \in \{1, 2, \dots, k\}} d_{ji}$;
- 7: 将样本 x_j 划入相应的簇: $C_{\lambda_j} = C_{\lambda_j} \cup \{x_j\}$;
- 8: **end for**
- 9: **for** $i = 1, 2, \dots, k$ **do**
- 10: 计算新均值向量: $\mu'_i = \frac{1}{|C_i|} \sum_{x \in C_i} x$;
- 11: **if** $\mu'_i \neq \mu_i$ **then**
- 12: 将当前均值向量 μ_i 更新为 μ'_i
- 13: **else**
- 14: 保持当前均值向量不变
- 15: **end if**
- 16: **end for**
- 17: **until** 当前均值向量均未更新

输出： 簇划分 $C = \{C_1, C_2, \dots, C_k\}$

优化目标：最小化平方误差 $E = \sum_{i=1}^K \sum_{x \in C_i} |x - \mu_i|^2$ ，其中 $\mu_i = \frac{1}{|C_i|} \sum_{x \in C_i} x$ 是簇 C_i 的均值向量

优点：计算高效，适合大规模数据

缺点：对初始中心敏感，可能陷入局部最优，需多次随机初始化。

2. GMM（高斯混合模型）

GMM 算法：高斯混合模型是一种基于概率模型的聚类算法，假设数据是由多个高斯分布的混合生成的。GMM 的核心思想是通过最大化似然函数，估计每个高斯分布的参数，从而实现聚类。

3. DBSCAN 算法

DBSCAN 算法：DBSCAN 是一种基于密度的聚类算法，通过寻找密度连通的区域来形成簇。DBSCAN 的核心思想是定义核心点、边界点和噪声点，通过扩展核心点来形成簇。

流程：

- (1) 定义两个参数： ϵ （邻域半径）和 MinPts（最小点数）。
- (2) 对于每个样本 $x^{(i)}$ ，计算其 ϵ -邻域内的样本数量。
- (3) 如果某个样本 ϵ -邻域内的样本数量大于或等于 MinPts，则该样本为核心点，将其邻域内的样本加入同一个簇。
- (4) 重复步骤(3)，直到所有核心点都被访问过。
- (5) 将未被访问的样本标记为噪声点。

优点：无需预设簇数，能发现任意形状簇，识别噪声点。双月模型

4. 层次聚类

层次聚类算法：层次聚类是一种基于树状结构的聚类算法，通过逐步合并或分割簇来形成层次结构。层次聚类的核心思想是通过计算簇之间的距离，逐步构建簇的层次关系。

流程：

- (1) 初始化每个样本为一个单独的簇。
- (2) 计算所有簇之间的距离，选择距离最近的两个簇进行合并。
- (3) 更新簇之间的距离，重复步骤(2)，直到所有样本合并为一个簇或达到指定的簇数量。

优点：它可以通过树状图直观地展示簇的层次关系，适用于需要逐步分析簇结构的情况。

5. 性能评价指标

(1) 外部指标

外部指标通过将聚类结果与某个“参考模型”（如专家标注的簇标签）进行比较来评估聚类性能

2.1 聚类性能的外部指标 (从“样本对”归属的角度)



对数据集 $D = \{x_1, x_2, \dots, x_m\}$, 假定通过聚类给出的簇划分为 $C = \{C_1, C_2, \dots, C_k\}$, 参考模型给出的簇划分为 $C^* = \{C_1^*, C_2^*, \dots, C_s^*\}$. 相应地, 令 λ 与 λ^* 分别表示与 C 和 C^* 对应的簇标记向量. 我们将样本两两配对考虑, 定义

$$a = |SS|, \quad SS = \{(x_i, x_j) \mid \lambda_i = \lambda_j, \lambda_i^* = \lambda_j^*, i < j\}, \quad (9.1)$$

$$b = |SD|, \quad SD = \{(x_i, x_j) \mid \lambda_i = \lambda_j, \lambda_i^* \neq \lambda_j^*, i < j\}, \quad (9.2)$$

$$c = |DS|, \quad DS = \{(x_i, x_j) \mid \lambda_i \neq \lambda_j, \lambda_i^* = \lambda_j^*, i < j\}, \quad (9.3)$$

$$d = |DD|, \quad DD = \{(x_i, x_j) \mid \lambda_i \neq \lambda_j, \lambda_i^* \neq \lambda_j^*, i < j\}, \quad (9.4)$$

其中集合 SS 包含了在 C 中隶属于相同簇且在 C^* 中也隶属于相同簇的样本对, 集合 SD 包含了在 C 中隶属于相同簇但在 C^* 中隶属于不同簇的样本对, ... 由于每个样本对 (x_i, x_j) ($i < j$) 仅能出现在一个集合中, 因此有 $a + b + c + d = m(m-1)/2$ 成立.

20

2.1 聚类性能的外部指标



- Jaccard系数 (Jaccard Index, Jaccard Similarity Coefficient)

$$JC = \frac{a}{a + b + c}$$

- FM指数 (Fowlkners & Mallows Index, FMI)

$$FMI = \sqrt{\frac{a}{a+b} \cdot \frac{a}{a+c}}$$

- Rand指数 (Rand Index, RI)

$$RI = \frac{2(a+d)}{m(m-1)}$$

		Groundtruth	
		同簇	异簇
Prediction	同簇	SS(a)	SD(b)
	异簇	DS(c)	DD(d)

21

(2) 内部指标

内部指标不依赖于外部的参考模型, 而是通过评估簇内的相似度和簇间的差异来评估聚类性能

簇内样本间平均距离: 衡量簇内样本的紧密程度, 值越小, 簇内样本越紧密, 聚类性能越好。

$$\text{avg}(C) = \frac{2}{|C|(|C|-1)} \sum_{1 \leq i < j \leq |C|} \text{dist}(c_i, c_j)$$

簇内样本间最远距离（直径）：衡量簇内样本的最大距离，值越小，簇内样本越紧密，聚类性能越好。

$$\text{diam}(C) = \max_{1 \leq i < j \leq |C|} \text{dist}(c_i, c_j)$$

簇间距离：衡量不同簇之间的距离，值越大，簇间差异越大，聚类性能越好

$$\text{dist}(C_i, C_j) = \min_{c_i \in C_i, c_j \in C_j} \text{dist}(c_i, c_j)$$

$$\text{dist}(C_i, C_j) = \text{dist}(\mu_i, \mu_j)$$

DB 指数：衡量簇内相似度与簇间差异的比值，值越小，聚类性能越好。

$$\text{DBI} = \frac{1}{k} \sum_{i=1}^k \max_{j \neq i} \left(\frac{\text{avg}(C_i) + \text{avg}(C_j)}{\text{dist}(\mu_i, \mu_j)} \right)$$

Dunn 指数：衡量簇间最小距离与簇内最大直径的比值，值越大，聚类性能越好。

$$\text{DI} = \frac{\min_{1 \leq i < j \leq k} \text{dist}(C_i, C_j)}{\max_{1 \leq i \leq k} \text{diam}(C_i)}$$

十一、机器学习理论

1. 偏差-方差理论与过拟合

(1) **偏差 (Bias)**: 偏差表示模型的预测值与真实值之间的差异。高偏差意味着模型对训练数据的拟合不足, 通常表现为欠拟合。

(2) **方差 (Variance)**: 方差表示模型在不同训练数据上的预测值之间的差异。高方差意味着模型对训练数据的拟合过于复杂, 通常表现为过拟合。

(3) **泛化误差**: 偏差、方差和噪声之和, $E(f; D) = Bias^2 + Variance + Noise^2$

(4) **过拟合**: 过拟合是指模型在训练数据上表现很好, 但在测试数据上表现很差。过拟合通常是因为模型过于复杂, 导致模型对训练数据的噪声和细节过度拟合

过拟合解决方案: 调整模型的复杂度、增加训练数据、正则化、早停、Dropout。

(5) **正则化**: 正则化是一种防止过拟合的技术, 通过在代价函数中加入正则化项来限制模型的复杂度。常见的正则化方法包括 L1 正则化 (Lasso) 和 L2 正则化 (岭回归)

L1 正则化: 在损失函数中添加所有权重的绝对值之和作为正则化项, 来限制模型的复杂度。L1 正则化的目标是使模型中的权重尽可能稀疏, 即让一些权重变为 0

假设原始损失函数为 $J(\theta)$, L1 正则化后的损失函数为: $J_{L1}(\theta) = J(\theta) + \lambda \sum_{i=1}^n |\theta_i|$

θ 是模型的权重向量, λ 是正则化参数, 控制正则化项的强度, n 是权重的数量

L2 正则化: 在损失函数中添加所有权重的平方和作为正则化项, 来限制模型的复杂度。L2 正则化的目标是使模型中的权重尽可能小, 但不会使权重变为 0。L2 正则化后的损失函数为: $J_{L2}(\theta) = J(\theta) + \lambda \sum_{i=1}^n \theta_i^2$

2. PAC 学习

PAC 辨识: 在给定的假设空间中, 找到一个假设 (模型), 使得该假设在高概率的情况下, 与真实模型的误差足够小

PAC 可学习: 如果存在一个学习算法, 能够在高概率的情况下, 找到一个近似正确的假设, 那么这个假设空间 H 就被称为 PAC 可学习的

弱学习和强学习算法的等价性: 任意给定仅比随机猜测略好的弱学习算法, 可以将其提升为强学习算法。弱学习器是指误差率略低于随机猜测的学习器, 强学习器是指误差率非常低的学习器。PAC 学习的一个重要结论是, 通过组合多个弱学习器, 可以构建一个强学习器。

例如, AdaBoost 算法就是基于 PAC 学习理论的一个成功应用

3. 统计学习理论

统计学习理论是一种基于统计学的机器学习理论，用于分析学习模型的泛化能力。统计学习理论的核心概念包括 VC 维、结构风险最小化等

(1) 模型的泛化能力：对于独立同分布的测试样本推断的能力。（模型能否被泛化到测试样本的推断上）

(2) VC 维：衡量模型复杂度（假设空间复杂度）的一个指标，表示模型能够打散的最大样本集的大小。VC 维越大，模型的复杂度越高，泛化能力越差。如二维线性分类器的 VC 维为 3（无法解决 4 个点的异或问题）。基于 VC 维的泛化误差界是分布无关、数据独立的。

(3) 结构风险最小化（SRM）：结构风险最小化是一种优化策略，通过最小化经验风险和置信风险的和，来提高模型的泛化能力。

SVM 是以 VC 维为核心的统计学习理论的代表性方法，通过最大化间隔（控制 VC 维，最小化泛化误差上界）和正则化，实现 SRM。

经验风险：模型在训练集上的平均损失，即“训练误差”

置信风险：由于训练数据有限，模型在未知数据上的表现与训练表现之间的差异

结构风险：通过控制模型复杂度来平衡经验风险和置信风险，避免过拟合

(4) 统计学习三要素：

模型：用于描述数据和预测目标之间关系的数学表达式或函数。在统计学习中，模型通常是一个假设空间，包含了所有可能的预测函数。模型的选择取决于具体问题的性质和数据的特点。

如线性回归模型（线性方程）、决策树模型（树状结构）、神经网络模型（一个网络）

策略：指根据什么准则来选择最优模型。在统计学习中，通常通过定义一个损失函数或代价函数来衡量模型的性能，并通过优化这个函数来选择最优模型。

如均方误差、交叉熵损失、结构风险最小化

算法：指具体的计算方法，用于从假设空间中选择最优模型。算法的选择取决于模型的复杂度和优化问题的性质。

如梯度下降法、牛顿法、共轭梯度法、SVM 的 SMO

4. 基本概念

假设空间：所有可能的假设集合。如在线性回归中假设空间是所有线性函数的集合。

归纳学习：归纳是从特殊到一般的泛化过程。既从具体的事实归结出一般性规律。从样例中学习，显然是一个归纳的过程，因此也称归纳学习。

从归纳的角度，假设空间可以是候选概念集合，归纳学习是从概念集合（假设空间）中找到与训练集匹配的概念或假设

归纳偏好：在假设空间中选择模型的准则或偏好。如奥卡姆剃刀（若有多个假设与观察一致，则选最简单的那一个）、最大间隔（SVM）。偏好可以帮助我们在假设空间中进行搜索，提高学习效率。

十二、Python 基本用法

1. Numpy 基本用法

参见第一次上机

2. scipy.optimize.minimize 最小化代价函数

第三次上机